# RECAP

- Last week
  - Superglobal variables
  - Passing values using GET/ POST
  - Compare GET vs. POST
  - Visual Diagram for Code Example



# **SESSION** HANDLING, CONDITIONAL STATEMENTS, LOOPS

## SERVER-SIDE WEB DEVELOPMENT - **5COSC012C.2**

# INTENDED LESSON OUTCOME

- At the end of the lesson students will be able to use PHP as a server-side scripting language for;
  - PHP Conditional Statements
  - PHP Loops
  - PHP Session variables
  - Session initialization
  - Clearing of a session

# TODAY'S TOPICS

- PHP Conditional Statements (IF ELSE)
- PHP Repetitive Statements (FOREACH)
- PHP Session variables
- Session initialization
- Clearing of a session

# PHP CONDITIONAL STATEMENTS

To perform different actions for different conditions, You can use conditional statements in your PHP code to do this.

- **if statement -** executes some code if one condition is true
- **if...else statement -** executes some code if a condition is true and another code if that condition is false
- **if...elseif....else statement -** executes different codes for more than two conditions
- **switch statement -** selects one of many blocks of code to be executed

# PHP - THE IF STATEMENT

The if statement executes some code if one condition is true.

Syntax

- if (*condition*) {
  *code to be executed if condition is true*;
  }

```
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
}
?>
```

# PHP - THE IF...ELSE STATEMENT

The if....else statement executes some code if a condition is true and another code if that condition is false.

```php
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

Syntax

if (*condition*) {
    *code to be executed if condition is true;*
} else {
    *code to be executed if condition is false;*
}

# PHP - THE IF...ELSEIF....ELSE STATEMENT

The if....elseif...else statement executes different codes for more than two conditions.

```
<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

Syntax

```
if (condition) {
    code to be executed if this condition is true;
} elseif (condition) {
    code to be executed if this condition is true;
} else {
    code to be executed if all conditions are false;
}
```

# THE PHP SWITCH STATEMENT

Use the switch statement to **select one of many blocks of code to be executed**.

Syntax

```
switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    case label3:
        code to be executed if n=label3;
        break;
    ...
    default:
        code to be executed if n is different from all labels;
}
```

# THE PHP SWITCH STATEMENT CONTD…

```
<?php
$favcolor = "red";
switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red,
blue, nor green!";
}
?>
```

# THE PHP FOR LOOP

The for loop is used when you know in advance how many times the script should run.

*Syntax*

```
for (init counter; test counter; increment counter) {
    code to be executed;
}
```

```
<?php
for ($x = 0; $x <= 10;
$x++) {
    echo "The number
is: $x <br>";
}
?>
```

Parameters:

- *init counter*: Initialize the loop counter value
- *test counter*: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends.
- *increment counter*: Increases the loop counter value

# THE PHP FOREACH LOOP

The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

*Syntax*

```
foreach ($array as $value) {
    code to be executed;
}
```

```
<?php
$colors = array("red",
"green", "blue", "yellow");

foreach ($colors as $value) {
    echo "$value <br>";
}
?>
```

For every loop iteration, the value of the current array element is assigned to $value and the array pointer is moved by one, until it reaches the last array element.

# BRAIN-BREAK

- **Cookies vs. Sessions**
  - What are cookies?
  - What are sessions?
  - Compare & Contrast!



# PHP 5 SESSIONS

- A session is a way to store information (in variables) to be used across multiple pages.
- Unlike a cookie, the information is not stored on the users computer.
- the web server does not know who you are or what you do, because the HTTP address doesn't maintain state
- Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.



**Tip:** If you need a permanent storage, you may want to store the data in a database.

# START A PHP SESSION INITIALIZATION

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>
</body>
</html>
```

- A session is started with the session_start() function.
- Session variables are set with the PHP global variable: $_SESSION.



**Note:** The session_start() function must be the very first thing in your document. Before any HTML tags.

# GET PHP SESSION VARIABLE VALUES

- Notice that session variables are not passed individually to each new page, instead they are retrieved from the session we open at the beginning of each page (session_start()).
- Also notice that all session variable values are stored in the global $_SESSION variable:

# GET PHP SESSION VARIABLE VALUES

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] .
".<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] .
".";
?>
</body>
</html>
```

# SESSION VARIABLE VALUES FOR A USER

To show all the session variable values for a user session is to run the following code:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
print_r($_SESSION);
?>
</body>
</html>
```



**How does it work? How does it know it's me?**

Most sessions set a user-key on the user's computer that looks something like this: 765487cf34ert8dede5a562e4f3a7e12. Then, when a session is opened on another page, it scans the computer for a user-key. If there is a match, it accesses that session, if not, it starts a new session.

# MODIFY A PHP SESSION VARIABLE

To change a session variable, just overwrite it:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// to change a session variable, just overwrite it
$_SESSION["favcolor"] = "yellow";
print_r($_SESSION);
?>
</body>
</html>
```

# DESTROY A PHP SESSION

To remove all global session variables and destroy the session, use session_unset() and session_destroy():

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// remove all session variables
session_unset();

// destroy the session
session_destroy();
?>
</body>
</html>
```

# BRAIN-BREAK

- Sessions (with cookies) vs.
  **Sessions (without cookies)**



# SUMMARY

- PHP Conditional Statements (IF ELSE)
- PHP Repetitive Statements (FOREACH)
- PHP Session variables
- Session initialization
- Clearing of a session

# JOURNEY SO FAR?

- https://goo.gl/forms/G8cpTI5V192oiTbx2

# NEXT LESSON

- Server-side validation, Error handling
- SQL Insert